

# Sistemas Distribuídos e Tolerantes a Falhas

## Relatório do Trabalho — Parte 2

### Nomes:

Lucas Neto Ferreira (00597912)

Jean Smaniotto Argoud (00275602)

### Descrição do Ambiente de Testes

---

**Sistema Operacional:** ambiente Unix-like simulado via Windows Subsystem for Linux 2 (WSL2), executando a distribuição Ubuntu 22.04.5 LTS (Kernel 6.6.87.2-microsoft-standard-WSL2).

**Hardware Utilizado:** processador Intel i5-1035G1, 8 GB de memória RAM.

**Interpretador:** Python 3.13, fazendo uso exclusivo das primitivas nativas da biblioteca socket (API UDP) e struct para serialização de dados na comunicação inter-processos.

### (A) Funcionamento do Algoritmo de Eleição de Líder Implementado

---

#### *Lógica do Algoritmo (Valentão)*

O critério de prioridade entre os processos é a própria porta em que cada servidor está escutando (passada como argumento na inicialização): quanto maior a porta, maior a prioridade do processo na eleição. O mecanismo de detecção de falha é baseado em heartbeat: enquanto for o líder, o servidor primário envia periodicamente, a cada HEARTBEAT\_INTERVALO (0,3 s), uma mensagem TIPO\_HEARTBEAT a todos os servidores secundários conhecidos (`_thread_heartbeat()`). Cada servidor secundário mantém uma thread de watchdog (`_thread_watchdog()`) que verifica continuamente se o intervalo desde o último heartbeat (ou desde a última mensagem de replicação recebida, que também atualiza esse timestamp) ultrapassou HEARTBEAT\_TIMEOUT (1 s). Caso ultrapasse, e não haja eleição em andamento, uma nova eleição é disparada.

O processo de eleição, implementado em `_iniciar_eleicao()`, segue os seguintes passos:

- O servidor que detecta a falha do líder envia uma mensagem TIPO\_ELEICAO (contendo sua própria porta) para todos os servidores conhecidos com porta maior que a sua.
- Caso não exista nenhum servidor de porta maior na lista de servidores conhecidos, o processo se autoproclama líder imediatamente, chamando `_tornarse_lider()`.
- Caso existam servidores de porta maior, o processo aguarda, com um timeout de TIMEOUT\_ACK\_ELEICAO (2 s), o recebimento de uma mensagem TIPO\_ACK\_ELEICAO de algum deles. Se o ACK chegar, o processo assume que um servidor de maior prioridade seguirá o processo eleitoral e apenas aguarda a mensagem de coordenador.
- Se o timeout expirar sem receber nenhum ACK, o processo assume que os servidores de maior prioridade estão indisponíveis e se autoproclama líder.
- Todo servidor que recebe uma mensagem TIPO\_ELEICAO de um processo com porta menor responde imediatamente com um TIPO\_ACK\_ELEICAO e, caso ainda não esteja participando de uma eleição própria, dispara sua própria eleição em uma nova thread — reproduzindo o comportamento clássico do algoritmo do valentão, em que processos de maior prioridade assumem o controle do processo eleitoral.
- Ao se tornar líder (`_tornarse_lider()`), o novo processo primário envia uma mensagem TIPO\_COORDENADOR a todos os demais servidores conhecidos e uma mensagem de redirecionamento (TIPO\_REDIRECIONAMENTO) a todos os clientes conhecidos na sua tabela replicada, além de iniciar sua própria thread de heartbeat e o socket de atendimento a clientes na porta de descoberta.

### ***Justificativa da Escolha***

O algoritmo do valentão foi escolhido por ser explicitamente exigido pela especificação do trabalho. Sua implementação sobre UDP é direta: não exige um anel lógico entre processos nem estruturas de coordenação adicionais, bastando que cada processo conheça o conjunto de servidores ativos e compare sua prioridade (porta) com a dos demais. O uso da porta como identificador evita a necessidade de um ID adicional, já que cada servidor já precisa de uma porta única para operar. O uso de timeouts para detectar ausência de resposta — tanto na espera por ACK de eleição quanto na detecção de líder offline via heartbeat — é adequado a um ambiente

sem garantias de entrega como o UDP, tolerando perda pontual de mensagens sem comprometer a convergência do algoritmo.

## **(B) Implementação da Replicação Passiva**

---

### ***Modelo de Replicação Passiva***

O sistema adota o modelo de backup com cópia primária. O servidor primário é o único que atende requisições de clientes: o socket de descoberta (`sock_discovery`, vinculado à porta fixa `PORTA_DESCOBERTA_SERVIDORES = 8080`) só é aberto e processado enquanto o servidor for o líder — verificação feita por `_sou_lider()` no laço `_thread_gerenciar_requisicoes()`. Os clientes descobrem esse endereço por broadcast e só passam a usar um endereço diferente ao receber explicitamente uma mensagem `TIPO_REDIRECIONAMENTO`, enviada pelo novo líder após uma eleição — garantindo que todos os clientes sempre utilizem a mesma cópia primária.

A cada requisição de soma processada com sucesso pelo líder (`TIPO_REQUISICAO`), as variáveis de estado (`total_sum` e `num_reqs`) e a tabela de clientes (com o último id de requisição atendido de cada um) são atualizadas e imediatamente propagadas a todos os servidores secundários conhecidos através de `replicar_dados()`, que envia uma mensagem `TIPO_REPLICACAO` com o estado completo (não apenas o delta). Os secundários recebem essa mensagem em `receber_replicacao()` e atualizam seu estado local de forma idempotente.

Além da replicação incremental a cada requisição, existe uma sincronização inicial: ao subir, um servidor secundário faz um broadcast (`TIPO_NOVO_SERVIDOR`) na porta de descoberta (`sincronizar_servidores()`); o primário responde com o estado completo atual no mesmo formato de replicação, permitindo que um servidor recém-iniciado (ou reiniciado após falha) entre em conjunto com o estado já consistente, sem depender de ter recebido todas as replicações incrementais anteriores.

### ***Sincronização e Concorrência — Desafios Encontrados***

Na implementação atual, cada servidor mantém apenas duas threads de longa duração relacionadas ao papel de líder (`_thread_heartbeat` e `_thread_gerenciar_requisicoes`) mais a `_thread_watchdog`, e o atendimento de requisições de clientes é feito de forma sequencial dentro

do laço de `_thread_gerenciar_requisicoes()`, sem criação de uma thread nova por requisição recebida e sem uso de `threading.Lock` — visto que, nessa estrutura, apenas essa única thread lê e escreve o estado compartilhado (`total_sum`, `num_reqs`, `clientes`), não havendo concorrência de fato sobre essas variáveis. Isso diverge do exigido pela especificação, que pede explicitamente uma thread por requisição recebida; para atender a esse requisito à risca, seria necessário disparar uma thread para cada requisição em `TIPO_REQUISICAO` e proteger o acesso a `total_sum`, `num_reqs` e `clientes` com um `threading.Lock`.

## **Descrição dos Problemas Encontrados e Limitações de Ambiente**

---

### ***Isolamento de Rede no WSL2***

Durante os testes práticos simulando a execução do sistema em nós distribuídos, observou-se restrição no tráfego de pacotes UDP em broadcast no Windows Subsystem for Linux (WSL2): por padrão, ele opera sob uma arquitetura de switch virtual interno (modo NAT), o que isola o tráfego de broadcast entre a máquina hospedeira e a rede física local. Como as mensagens de descoberta (`TIPO_DESCOBERTA`, `TIPO_NOVO_SERVIDOR`) e os anúncios de coordenador (`TIPO_COORDENADOR`) dependem de broadcast ou de endereços obtidos via broadcast, essa barreira dificultava a convergência de nós rodando em máquinas WSL2 diferentes.

### ***Abordagem Adotada e Validação de Escopo***

Diante das limitações de rede impostas pelo ambiente de desenvolvimento WSL2, e da impossibilidade de provisionar hardware com Linux nativo para testes inter-máquinas, o escopo de validação do projeto foi delimitado. O sistema foi homologado executando múltiplos processos (clientes e servidores) na mesma instância WSL2, onde o tráfego local atende aos requisitos de broadcast do sistema. Assumimos que, em um ambiente operacional composto por servidores Linux nativos interconectados em uma rede local física (LAN), o algoritmo do valentão e o fluxo de replicação passiva funcionarão conforme especificado, uma vez que as barreiras de virtualização observadas no WSL2 não estarão presentes.

### ***Reenvio de Requisições e Erros de Conexão no Cliente***

Como o cliente usa um socket UDP não conectado, ao continuar enviando pacotes para o endereço do antigo líder (agora indisponível) antes do redirecionamento chegar, o sistema

operacional pode devolver um erro de conexão recusada no `recvfrom()` seguinte. O cliente trata esse caso em `enviar_com_confirmacao()`: ao capturar `ConnectionResetError`, evita reenviar a mesma requisição na iteração seguinte e aguarda a chegada do redirecionamento; ao processar um `TIPO_REDIRECIONAMENTO`, ele também drena, de forma não bloqueante, eventuais erros ou pacotes pendentes do servidor antigo antes de seguir operando normalmente com o novo líder.

### ***Replicação sem Confirmação***

A propagação de estado aos servidores secundários é feita via UDP sem confirmação de recebimento (fire-and-forget). Isso significa que uma mensagem de replicação pode se perder e um secundário pode ficar temporariamente desatualizado em relação ao primário. Esse risco é mitigado pelo fato de cada mensagem de replicação carregar o estado completo (e não um delta), de forma que a próxima replicação recebida corretamente já corrige qualquer inconsistência anterior — mas, no intervalo entre falhas, existe uma janela de inconsistência que é uma limitação conhecida da implementação.